Sleeping with One Eye Open: Fast, Sustainable Storage with Sandman

Yanbo Zhou, Erci Xu†, Anisa Su*, Jim Harris*, Adam Manzanares*, Steven Swanson

UC San Diego

†Shanghai Jiao Tong University

*Samsung Semiconductor

안녕하세요

Btw, this means hello in Korean.
At least that’s what ChatGPT said, *the ground truth of everything today*.

The Dilemma in Storage Evolution

Evolve towards high-performance and high-density all-flash servers

High performance

PCIe 5.0

NVMe SSDs

E.g., Samsung PM1743

Up to 2500K IOPS and 14GB/s BW

**Images and data from their official websites*

The Dilemma in Storage Evolution

Evolve towards high-performance and high-density all-flash servers

High performance

PCIe 5.0

NVMe SSDs

E.g., Samsung PM1743

Up to 2500K IOPS and 14GB/s BW

High density

Multi-layer (196)

TLC/QLC (3/4 bits / cell)

E.g., Solidigm P5336

Up to 122TB capacity per single drive

**Images and data from their official websites*

The Dilemma in Storage Evolution

Evolve towards high-performance and high-density all-flash servers

High performance

PCIe 5.0

NVMe SSDs

E.g., Samsung PM1743

Up to 2500K IOPS and 14GB/s BW

High density

Multi-layer (196)

TLC/QLC (3/4 bits / cell)

E.g., Solidigm P5336

Up to 122TB capacity per single drive

Disaggregated

High performance

RDMA NICs

E.g., NVIDIA CX-7

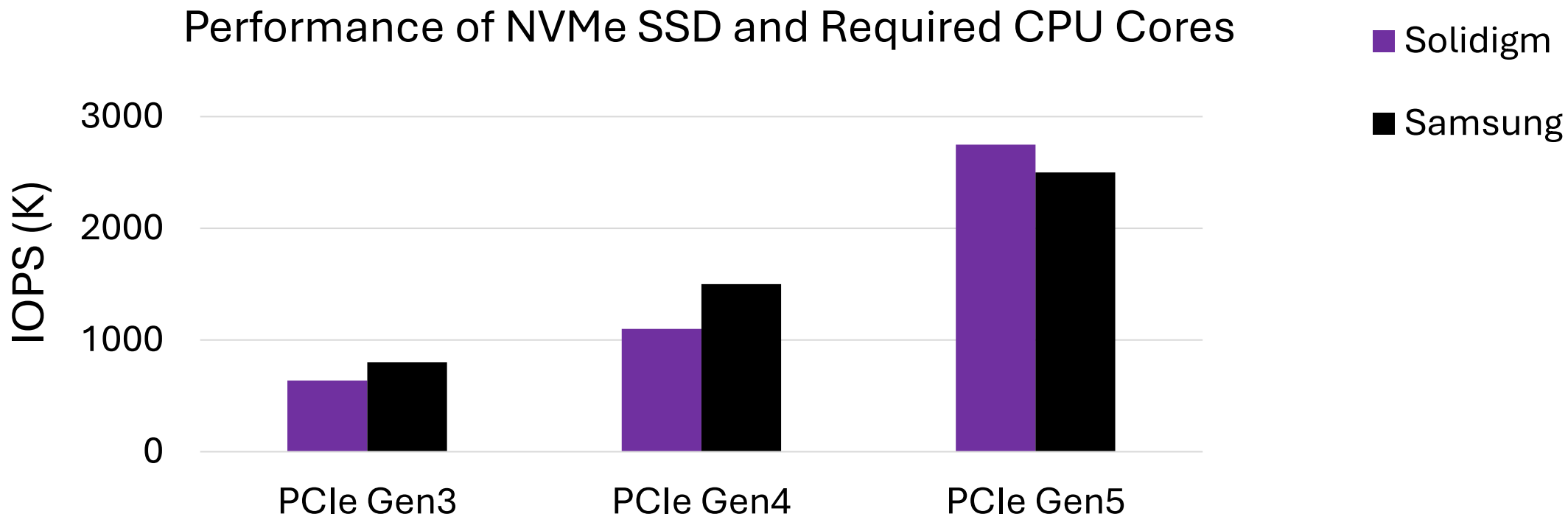
Up to 400Gbps and four network ports

**Images and data from their official websites*

The Dilemma in Storage Evolution

Reason 1: high demand for processing power (i.e., more CPU cores)

- SSD performance is almost doubled across PCIe generations

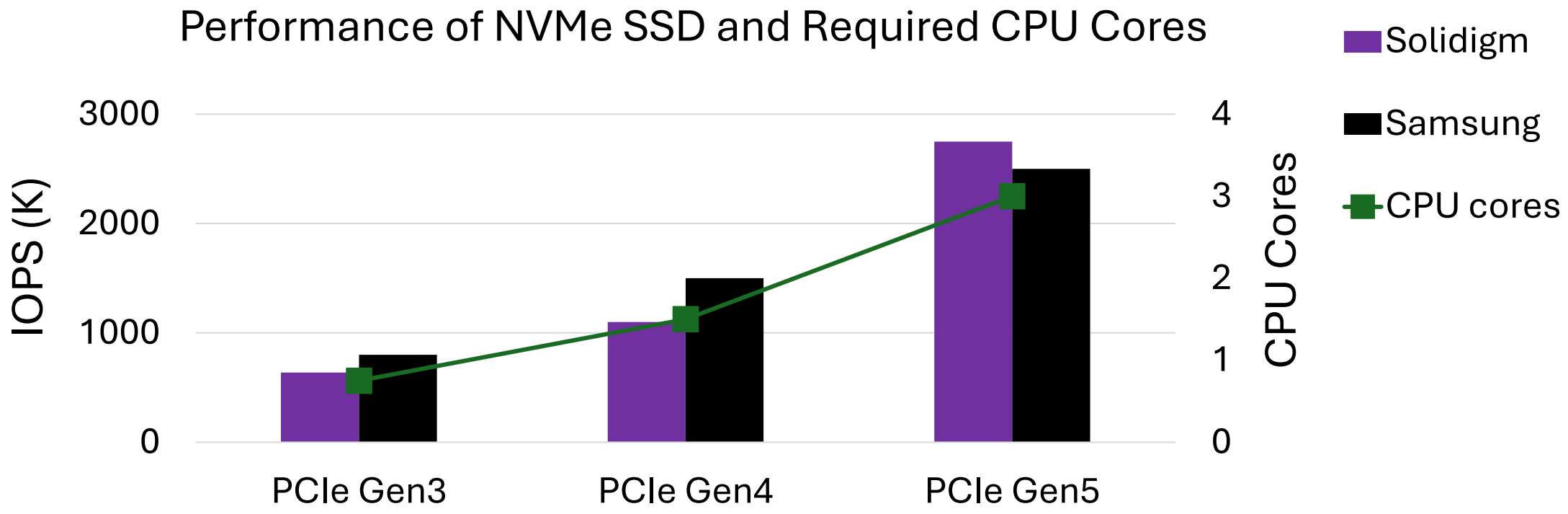


**Performance data are from official specifications and tested on SPDK by us*

The Dilemma in Storage Evolution

Reason 1: high demand for processing power (i.e., more CPU cores)

- SSD performance is almost doubled across PCIe generations
- Demand more CPU cores in host server to support such high performance



*Performance data are from official specifications and tested on SPDK by us

The Dilemma in Storage Evolution

Reason 2: high power consumption from processing I/Os (i.e., busy-polling)

1. SSD can adaptively change power based on workloads

	System Idle		Light Load		Peak Load	
	Power	Pct.	Power	Pct.	Power	Pct.
CPU	134 W	33%	244 W	34%	245 W	26%
SSD	74 W	17%	74 W	10%	296 W	32%
Others	204 W	50%	398 W	56%	399 W	42%
System	412 W	100%	716 W	100%	940 W	100%

SSD power (5W-25W) goes up to 25W only when it comes to 100% performance utilization and GC is running to erase data;

**SSD power is captured through upcoming NVMe TP4199 feature*

The Dilemma in Storage Evolution

Reason 2: high power consumption from processing I/Os (i.e., busy-polling)

- Require CPU cores to operate in busy-polling state (100% util.) at high frequency

1. SSD can adaptively change power based on workloads

	System Idle		Light Load		Peak Load	
	Power	Pct.	Power	Pct.	Power	Pct.
CPU	134 W	33%	244 W	34%	245 W	26%
SSD	74 W	17%	74 W	10%	296 W	32%
Others	204 W	50%	398 W	56%	399 W	42%
System	412 W	100%	716 W	100%	940 W	100%

2. CPU consumes high power consumption regardless of load.

SSD power (5W-25W) goes up to 25W only when it comes to 100% performance utilization and GC is running to erase data;

**SSD power is captured through upcoming NVMe TP4199 feature*

The Dilemma in Storage Evolution

Reason 2: high power consumption from processing I/Os (i.e., busy-polling)

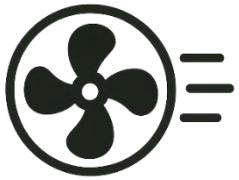
- Require CPU cores to operate in busy-polling state (100% util.) at high frequency

1. SSD can adaptively change power based on workloads

	System Idle		Light Load		Peak Load	
	Power	Pct.	Power	Pct.	Power	Pct.
CPU	134 W	33%	244 W	34%	245 W	26%
SSD	74 W	17%	74 W	10%	296 W	32%
Others	204 W	50%	398 W	56%	399 W	42%
System	412 W	100%	716 W	100%	940 W	100%

2. CPU consumes high power consumption regardless of load.

3. Lead to high collateral power consumption from other components



SSD power (5W-25W) goes up to 25W only when it comes to 100% performance utilization and GC is running to erase data;

**SSD power is captured through upcoming NVMe TP4199 feature*

The Dilemma in Storage Evolution

Reason 2: high power consumption from processing I/Os (i.e., busy-polling)

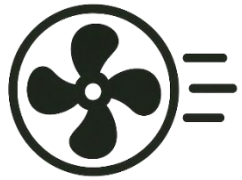
- Require CPU cores to operate in busy-polling state (100% util.) at high frequency

1. SSD can adaptively change power based on workloads

	System Idle		Light Load		Peak Load	
	Power	Pct.	Power	Pct.	Power	Pct.
CPU	134 W	33%	244 W	34%	245 W	26%
SSD	74 W	17%	74 W	10%	296 W	32%
Others	204 W	50%	398 W	56%	399 W	42%
System	412 W	100%	716 W	100%	940 W	100%

2. CPU consumes high power consumption regardless of load.

3. Lead to high collateral power consumption from other components



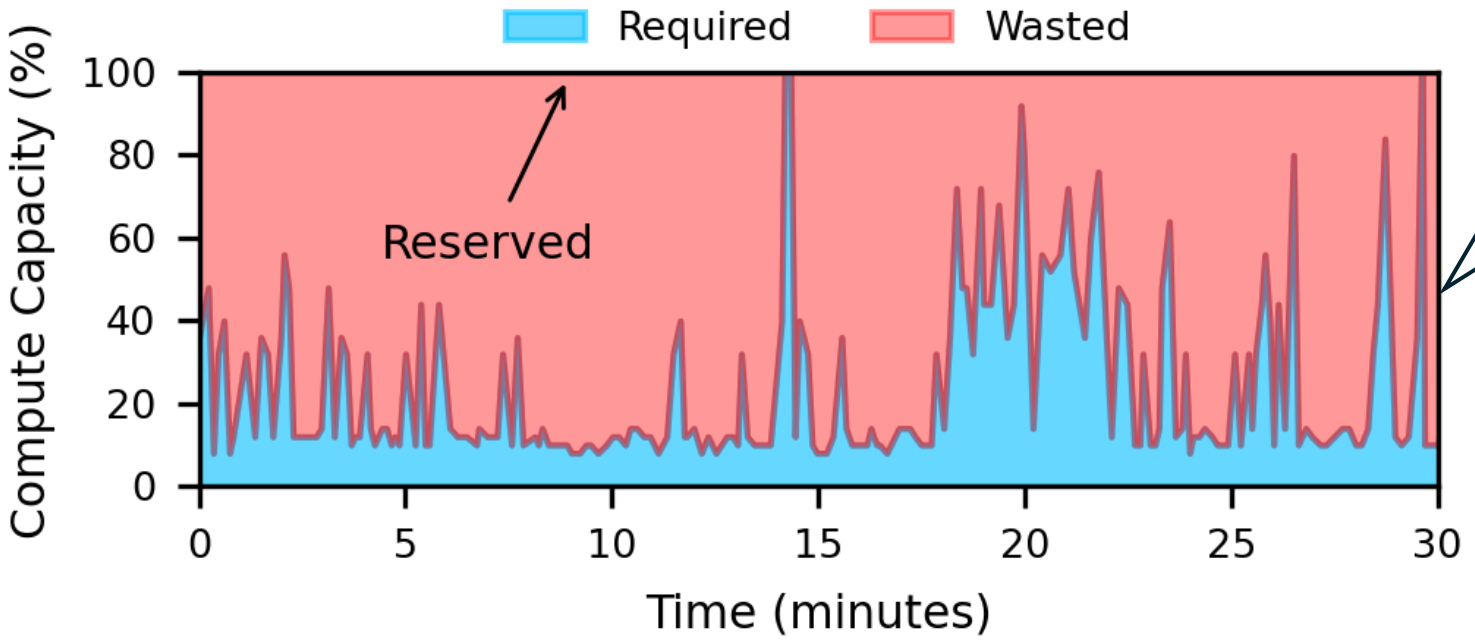
SSD power (5W-25W) goes up to 25W only when it comes to 100% performance utilization and GC is running to erase data;

**SSD power is captured through upcoming NVMe TP4199 feature*

The Dilemma in Storage Evolution

Reason 3: frequent variances and bursts in the field.

- High power consumption is acceptable if I/O pressure is consistently heavy, but…
- Hardware advancements have made bursts more severe and short-lived



Overprovisioning polling cores improves performance but waste lots of energy

**Data are from real cluster traces of Alibaba Cloud*

Measurement Study

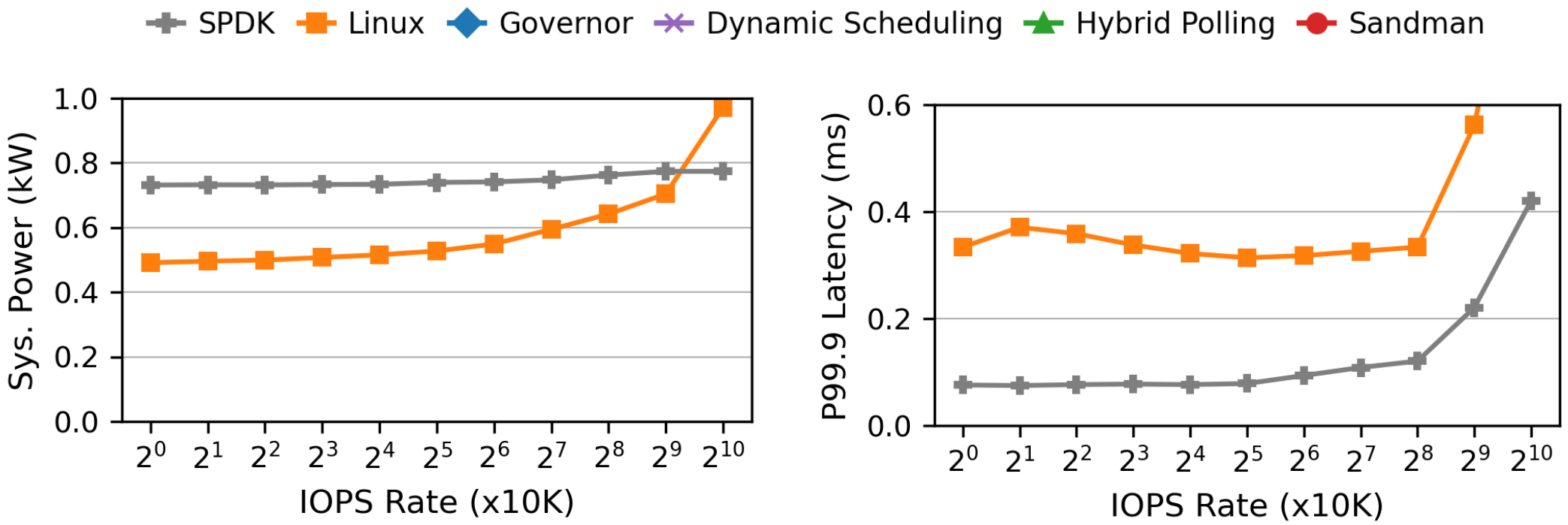
Ideal: comparable performance to busy polling with lower power consumption.

1. **SPDK**: busy-polling – best performance case
2. **Linux**: classic interrupts
3. **Governor**: adapting frequency of polling cores to workloads
4. **Dynamic Scheduling**: reallocates threads among polling cores.
5. **Hybrid Polling**: alternates states of polling cores between sleep and active states.

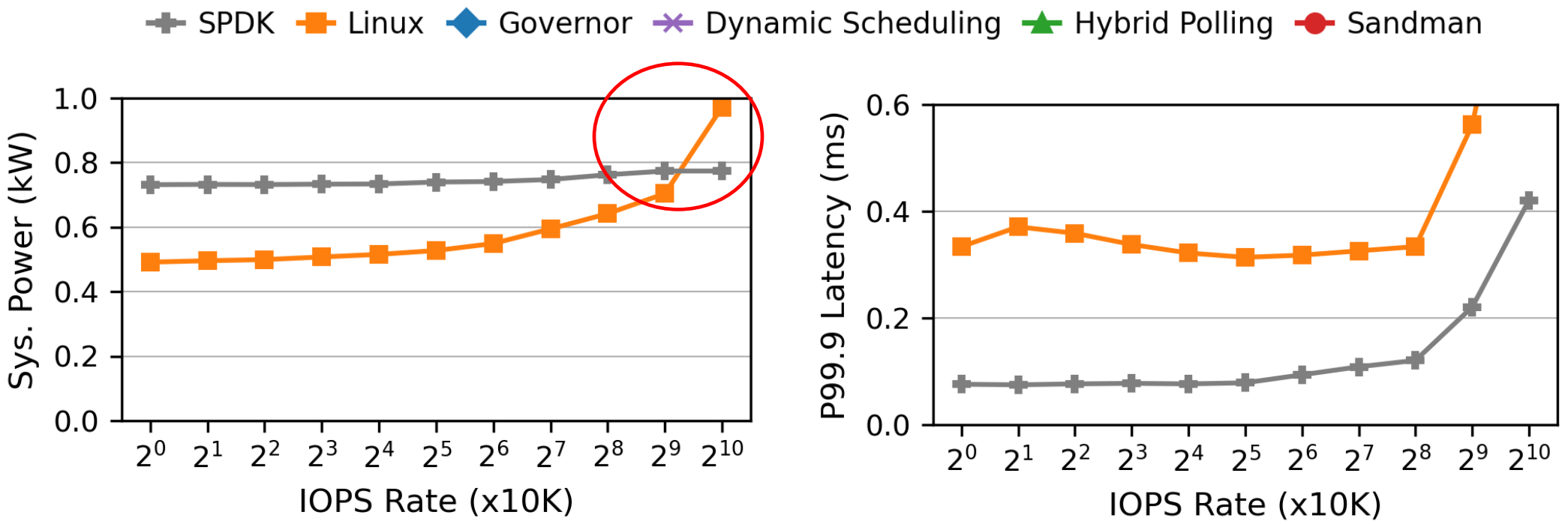
Workloads: 1) stable workloads at different IOPS rates and 2) burst (1s) workloads

Setup: two nodes connected via NVMeoF (16x PM1743 SSDs and RDMA NICs with 2x 200Gbps ports)

Stable Workloads – Linux Interrupts

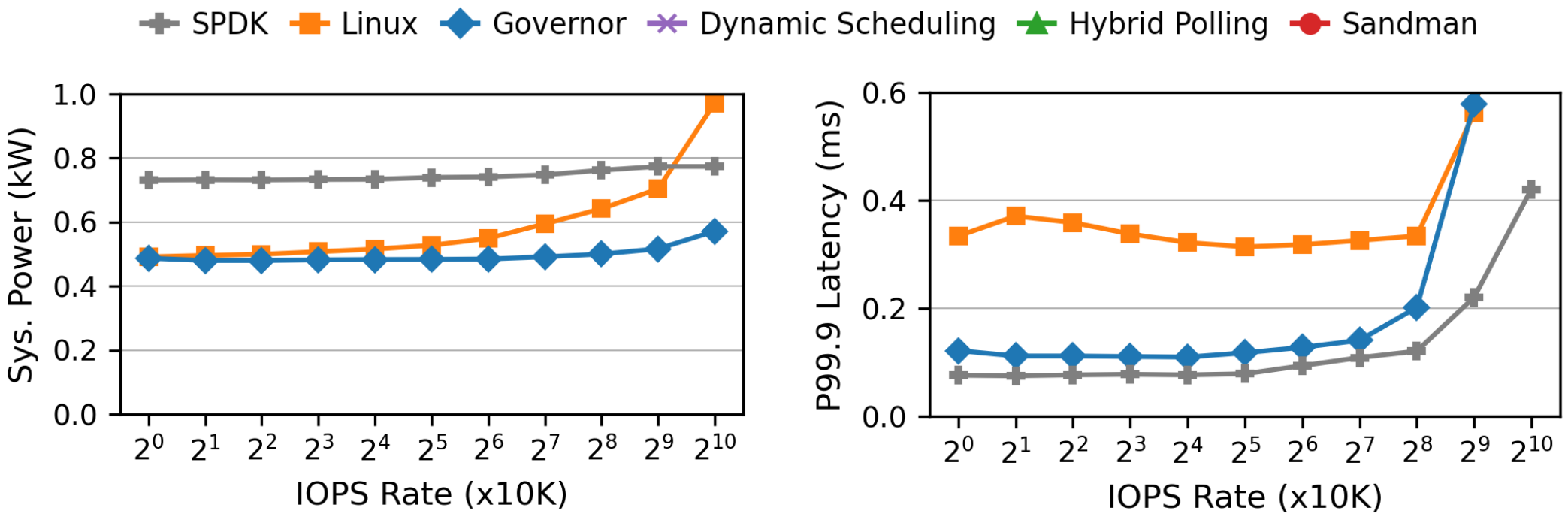


Stable Workloads – Linux Interrupts

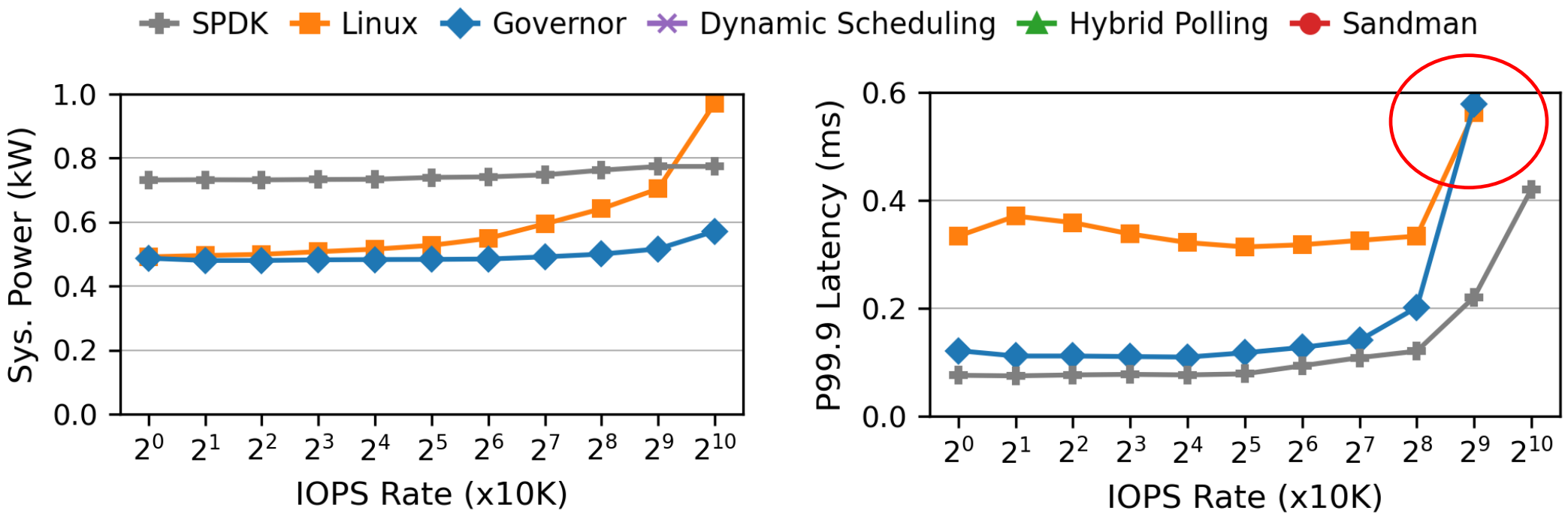


Observation: Interrupts show lower performance and can even consume ***higher-than-polling* energy** due to context switches under heavy workloads

Stable Workloads – Governor Frequency Scaling

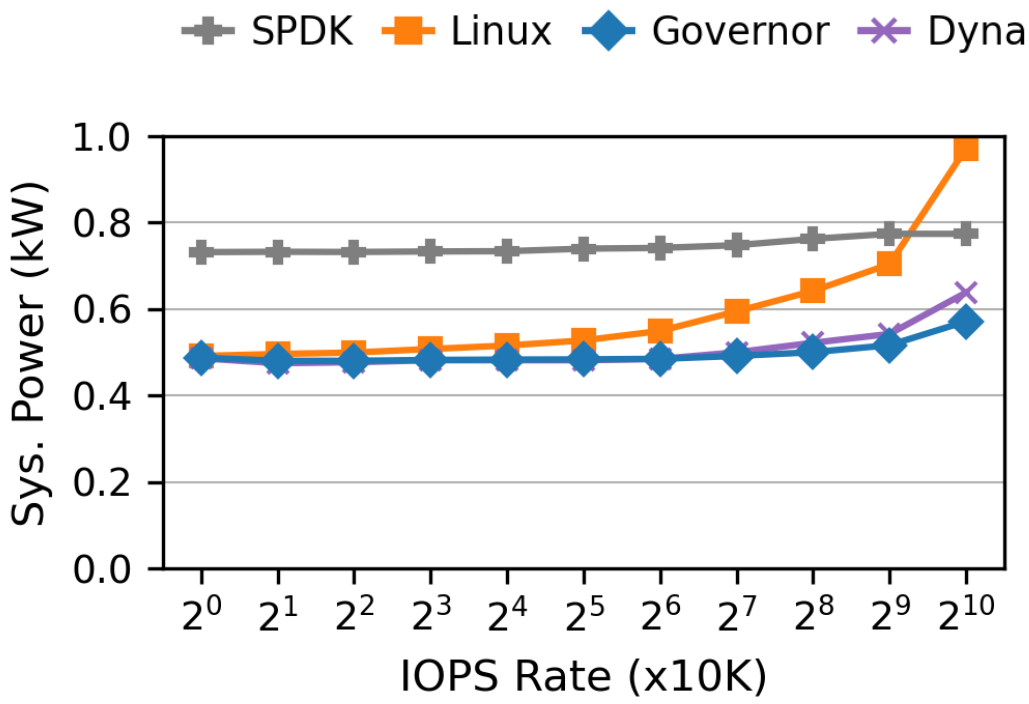


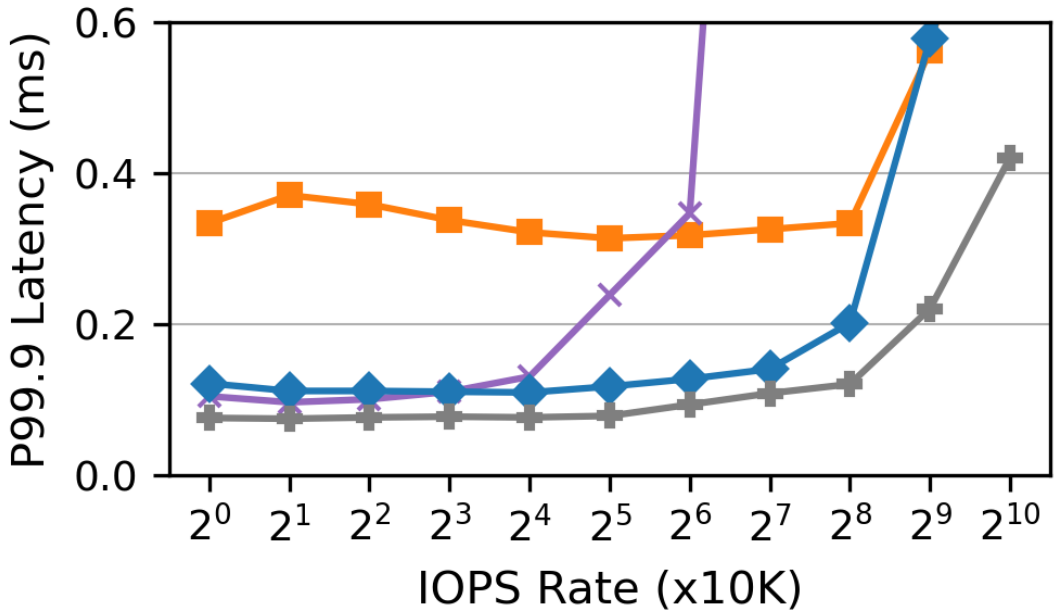
Stable Workloads – Governor Frequency Scaling



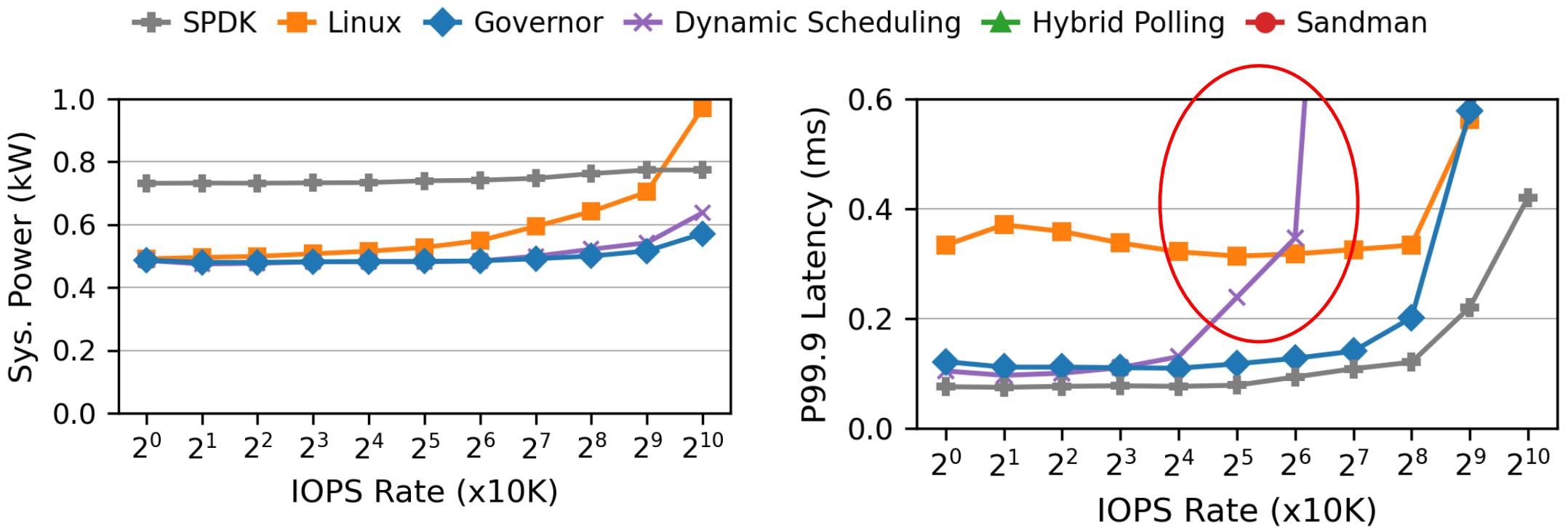
Observation: Governor reduces power consumption yet **increases tail latency** than busy-polling due to using low-frequency cores for processing

Stable Workloads – Dynamic Scheduling



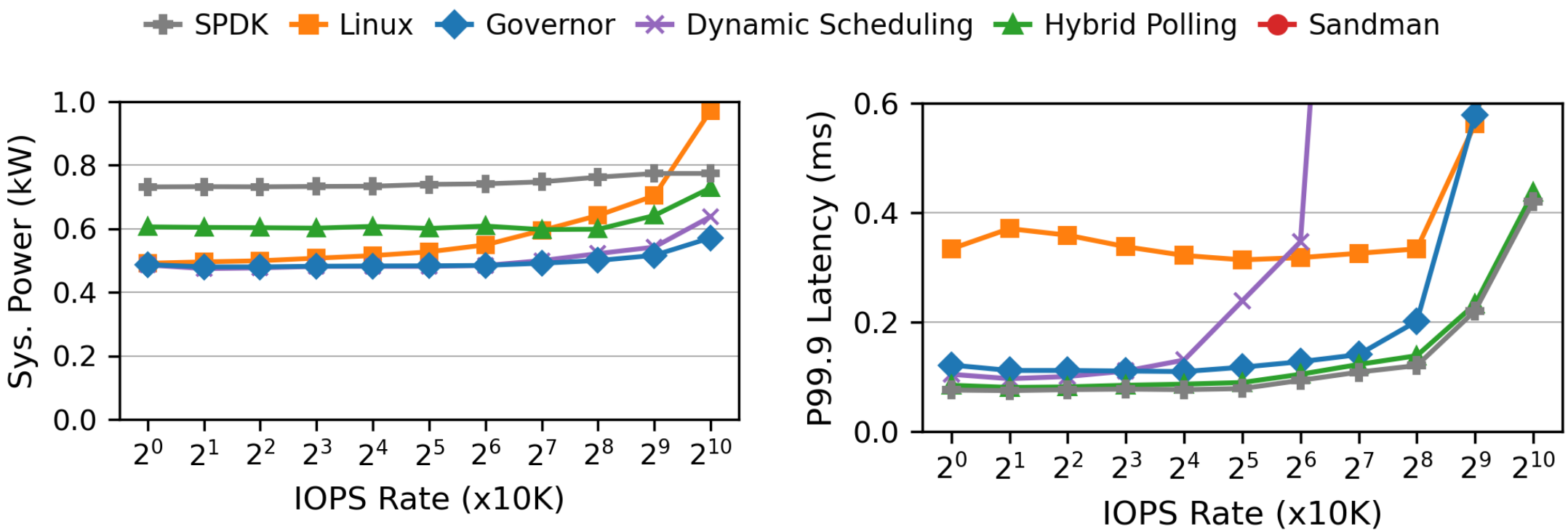


Stable Workloads – Dynamic Scheduling

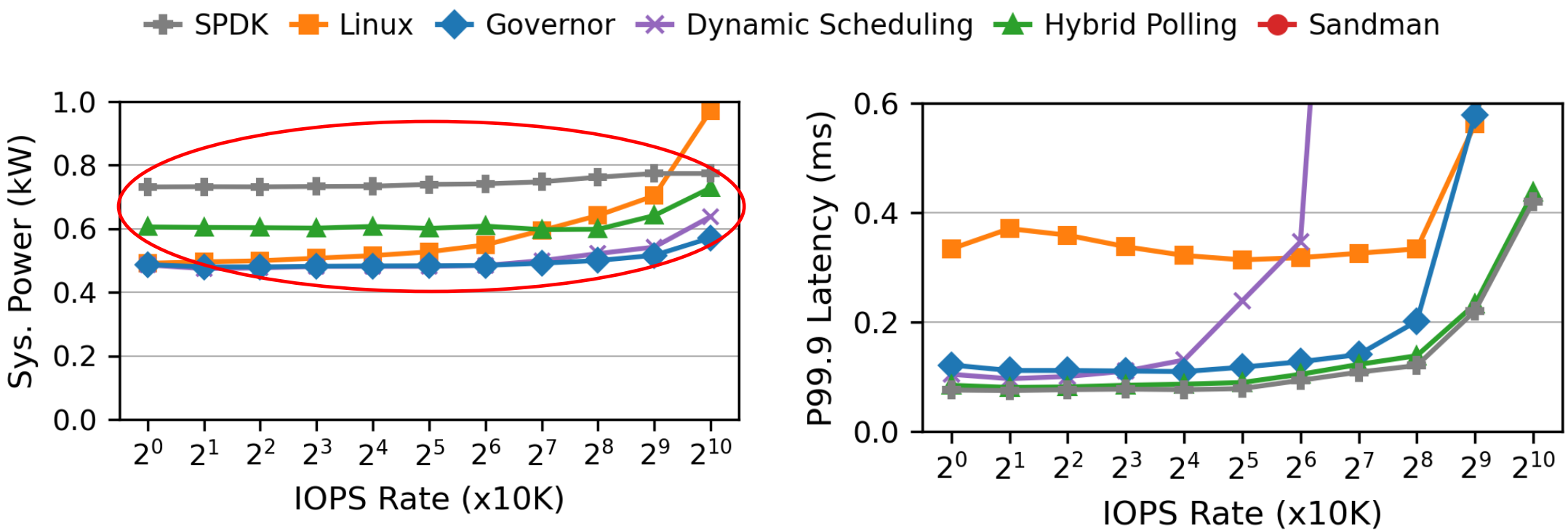


Observation: Adapting workload changes needs **microsecond-level detection**, incurring **frequent reallocation of tasks among cores.**

Stable Workloads – Hybrid Polling



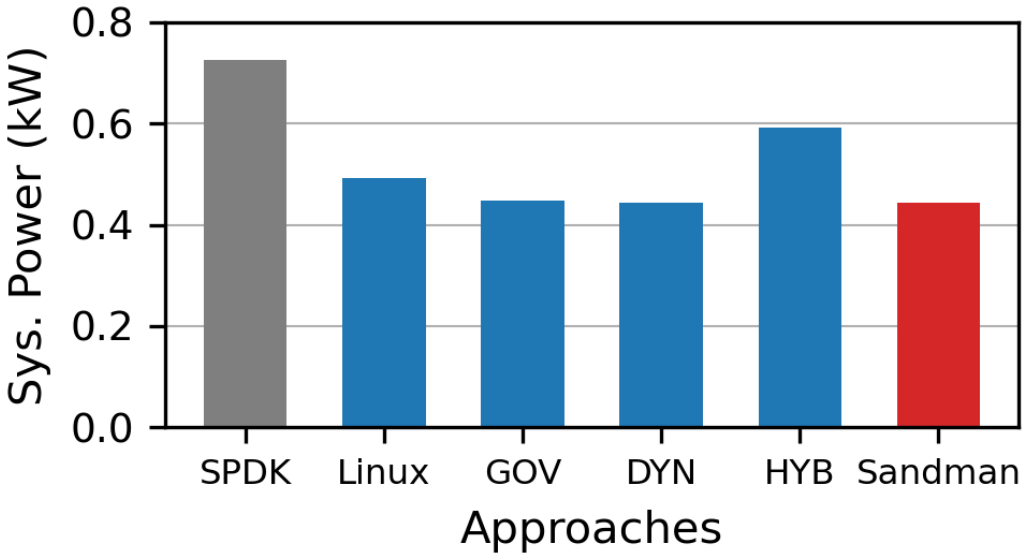
Stable Workloads – Hybrid Polling

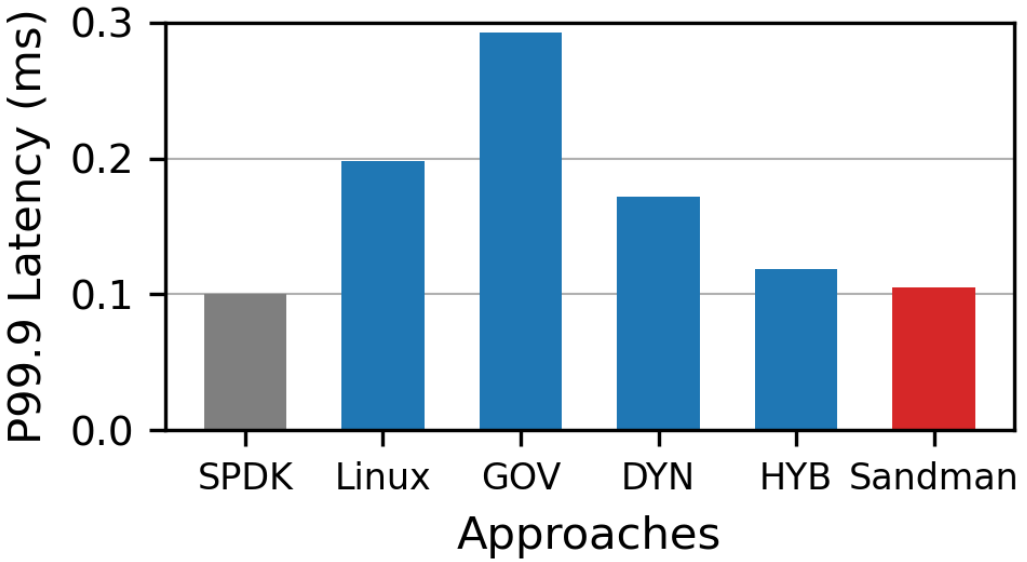


Observation: **Frequent transitions limit power saving**. Frequent transitions between low-power and high-performance states prevent CPU cores from staying in energy-saving mode for longer durations.

Burst Workloads

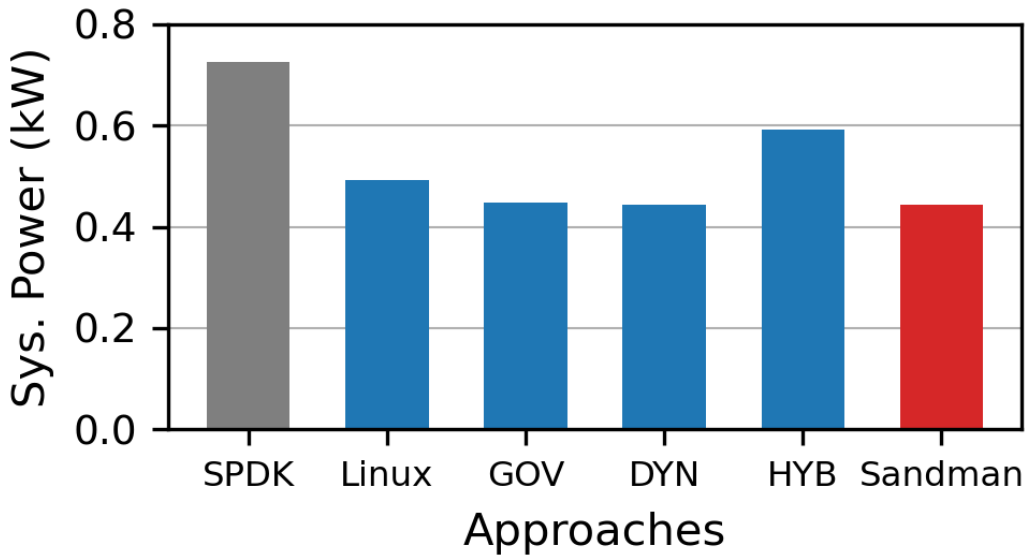
We expect scheduler scales compute resource ASAP under short-term bursts

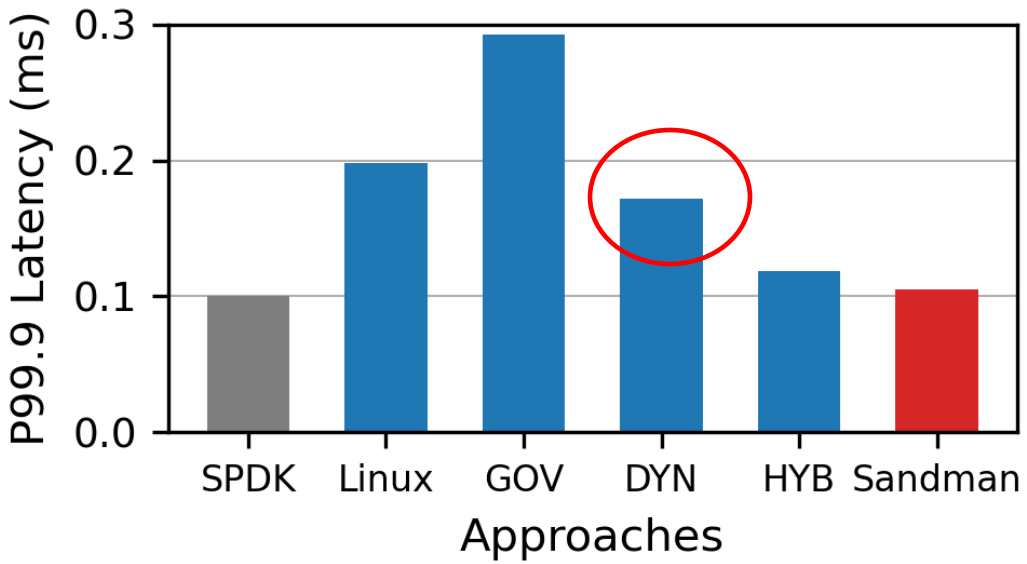




Burst Workloads

We expect scheduler scales compute resource ASAP under short-term bursts

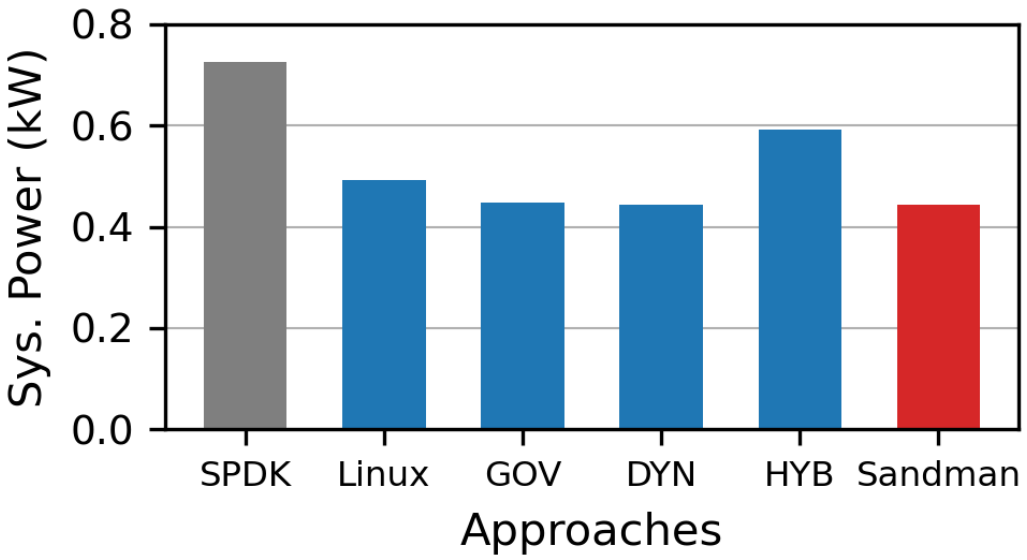


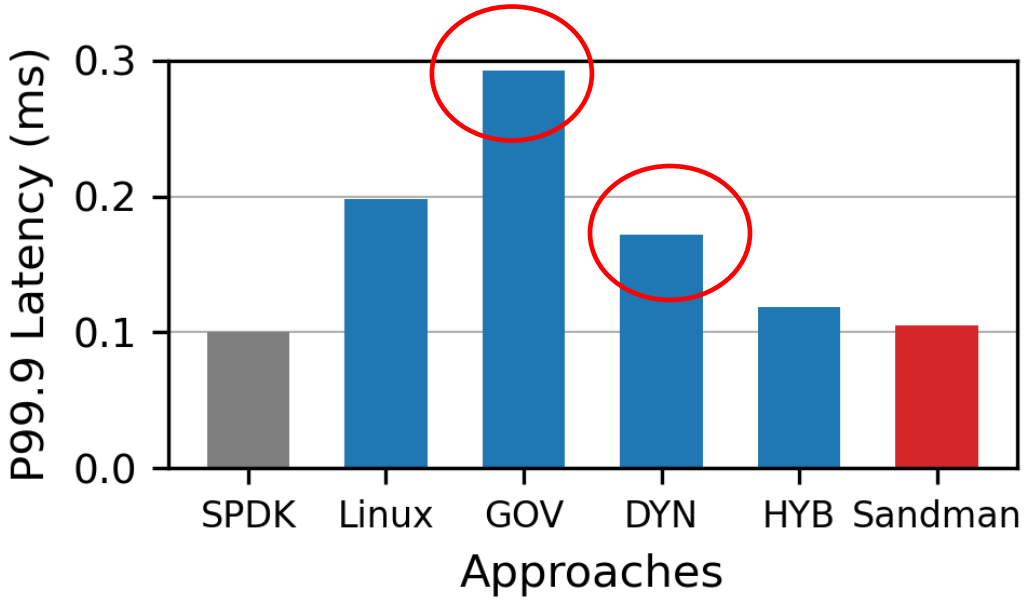


Observation: Dynamic scheduling wakes up sleeping cores via **software-based interrupts**, further leading to high tail latency

Burst Workloads

We expect scheduler scales compute resource ASAP under short-term bursts





Observation: Adjusting cores frequency takes time.

Warming up from low-power to high-power state takes **~450us.**

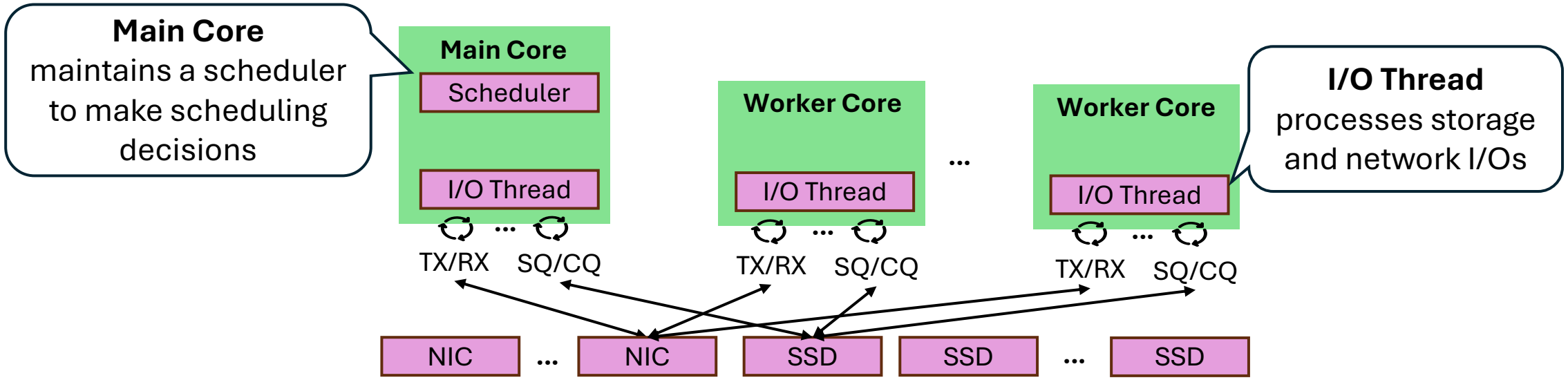
Challenges and Design Guidelines

Challenges	Guidelines
Changing core from low to high power state takes time	Shadow sleep cores NOT lower their frequency
Wake up core via SW-based interrupts → high latency	Avoid system calls/interrupts for waking up cores
Frequent transitions from sleep to polling limit power saving	Schedule sibling HT-cores together for longer sleep
Unprecise workload estimation leads frequent task reallocation	Measure short-term bursts from network queues

Sandman: Scheduler for Fast, Sustainable Storage

System diagram --- **Fundamental Architecture**

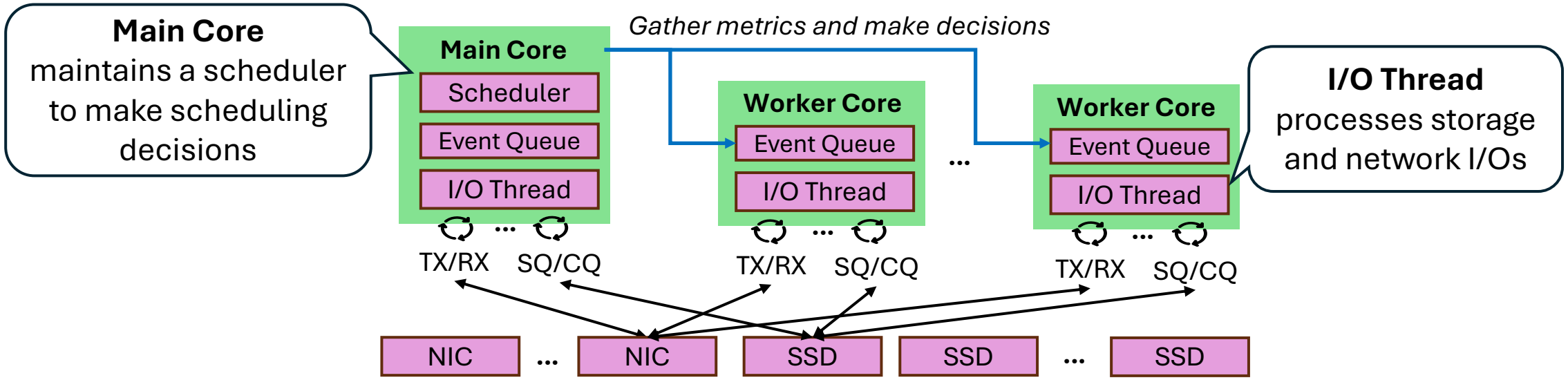
- Reserve CPU cores as ***worker cores*** and a ***main core*** running in polling mode



Sandman: Scheduler for Fast, Sustainable Storage

System diagram --- **Fundamental Architecture**

- Reserve CPU cores as ***worker cores*** and a ***main core*** running in polling mode
- Core-to-core communicate through ***event queues*** by sending event messages



Fast Resource Scaling Mechanism

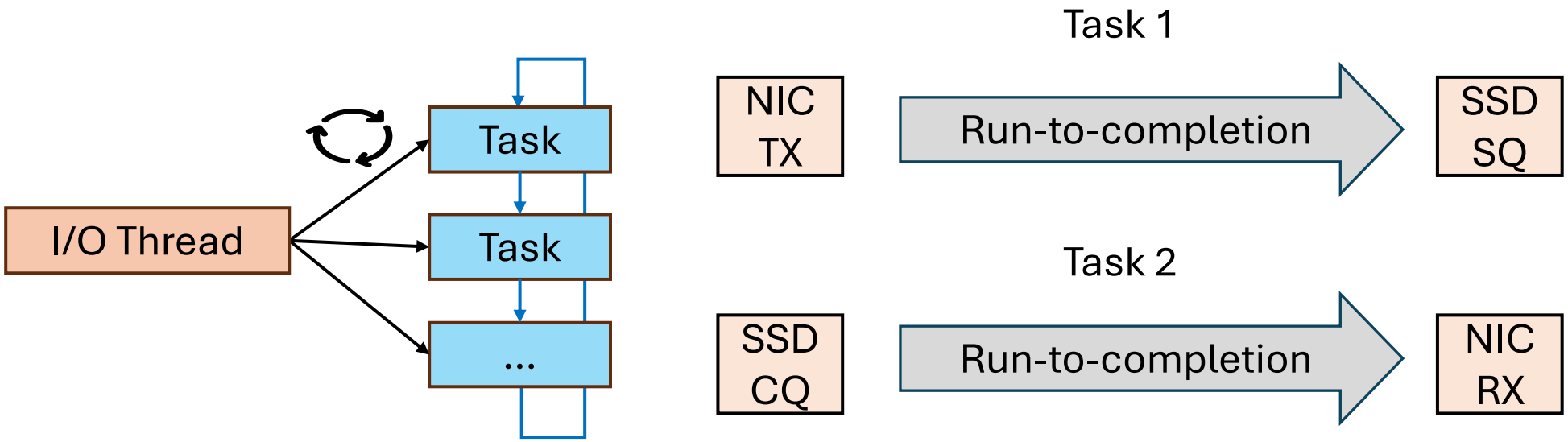
1. Lightweight I/O threads --- **Abstraction of Running Unit**

- **Goal**: move resource fast without overhead

Fast Resource Scaling Mechanism

1. Lightweight I/O threads --- **Abstraction of Running Unit**

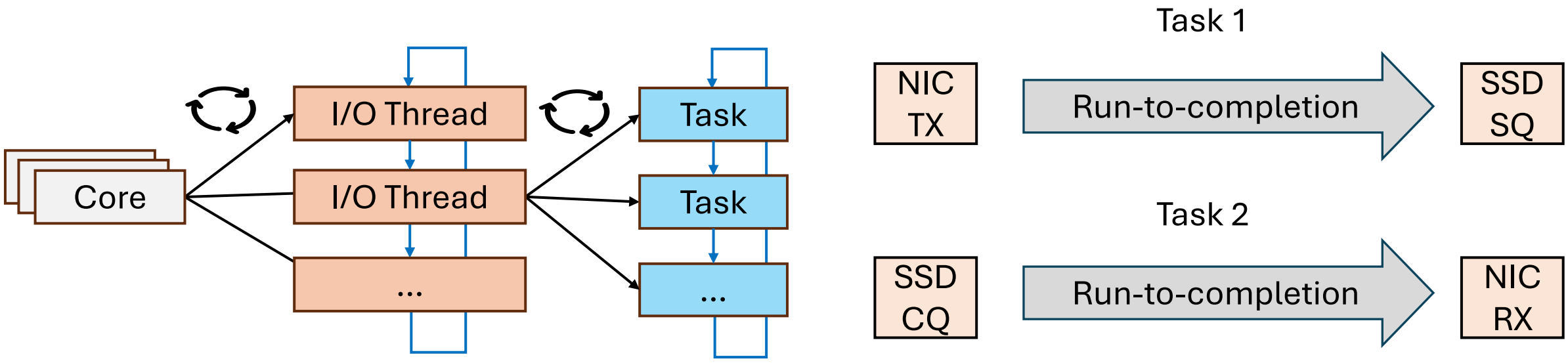
- **Goal**: move resource fast without overhead
- **Thread**: an abstraction of a set of tasks



Fast Resource Scaling Mechanism

1. Lightweight I/O threads --- **Abstraction of Running Unit**

- **Goal**: move resource fast without overhead
- **Thread**: an abstraction of a set of tasks
- Migration: removing a thread from the core's list and insert to another core's list



Fast Resource Scaling Mechanism

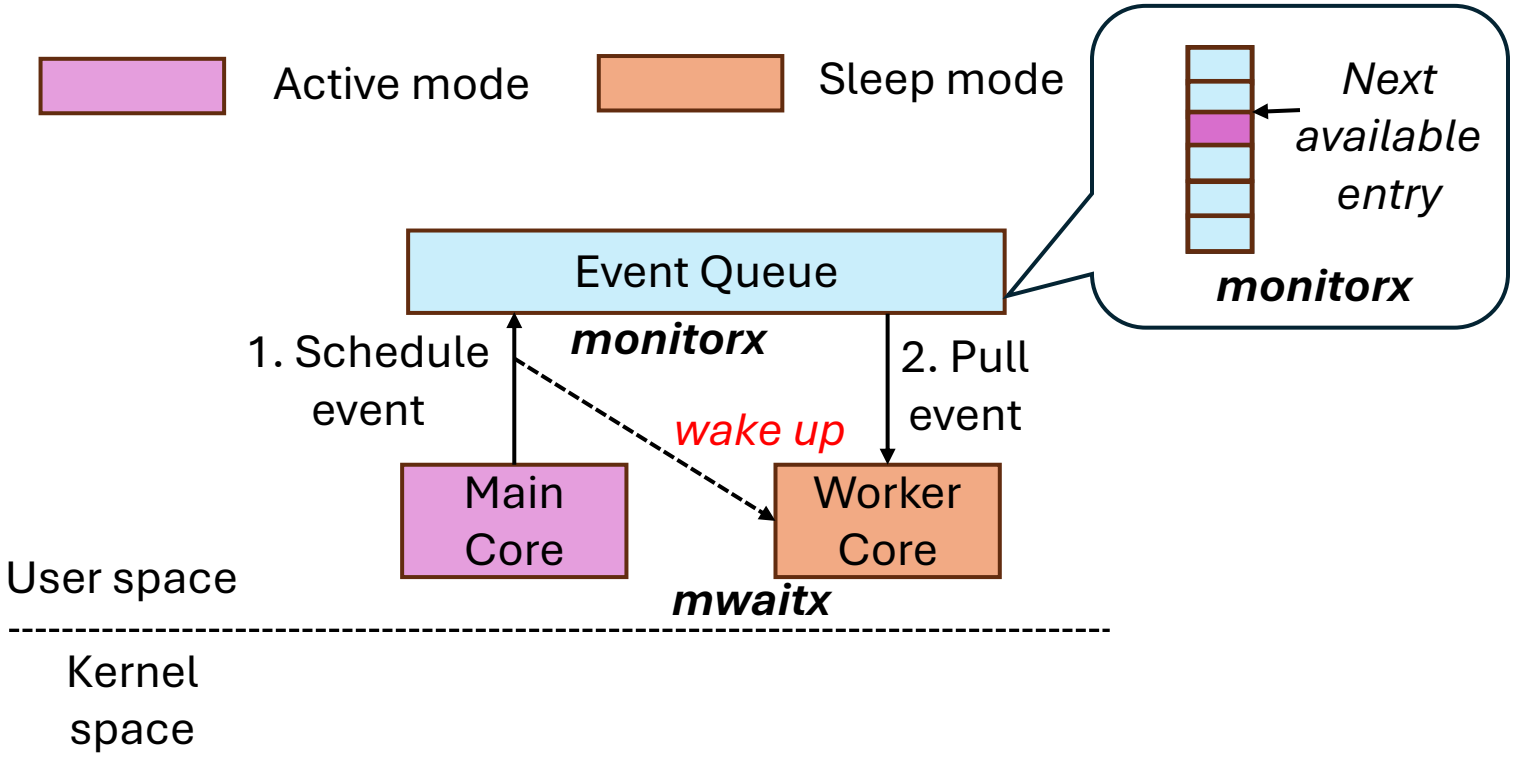
2. Shallow sleep with instant transition --- **How core enters/exits sleep?**

- **Goal**: wake up fast without overhead

Fast Resource Scaling Mechanism

2. Shallow sleep with instant transition --- **How core enters/exits sleep?**

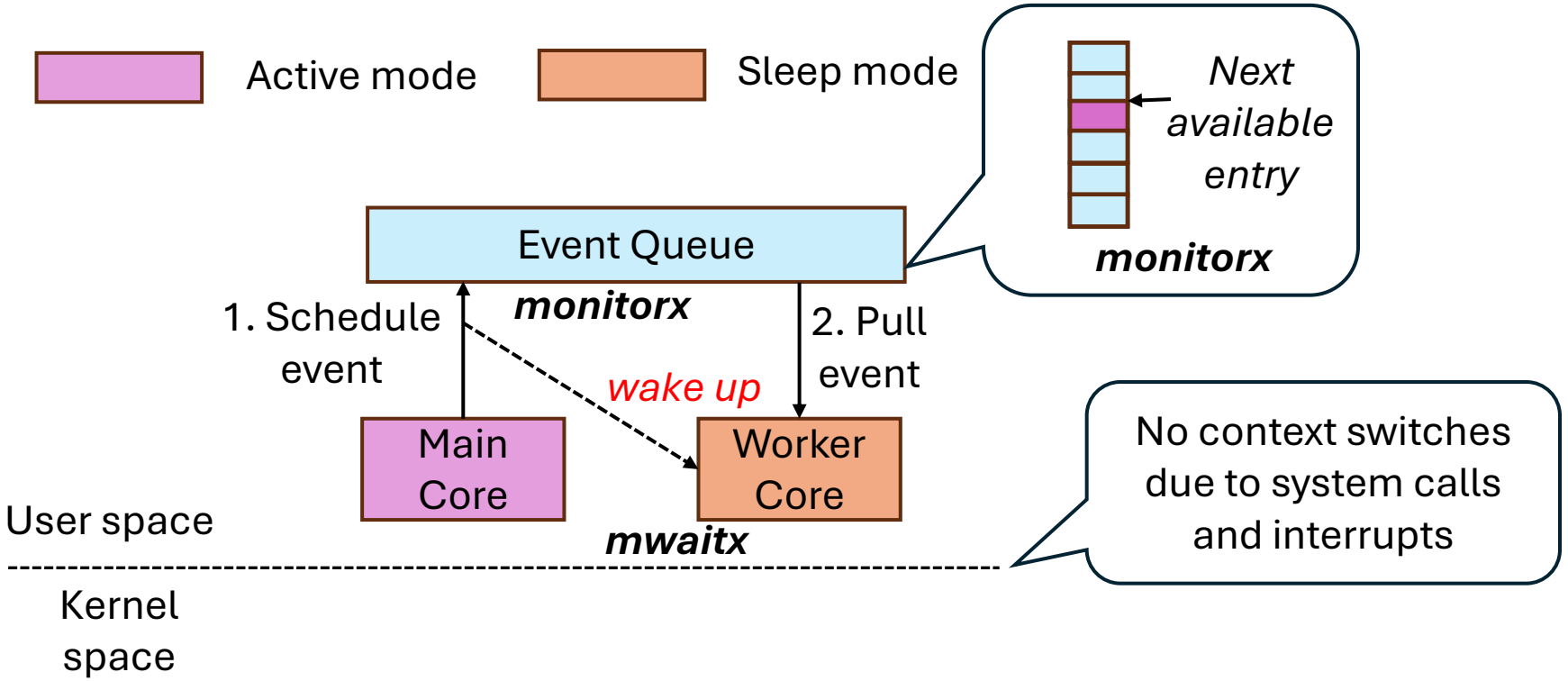
- **Goal**: wake up fast without overhead
- Wake up core based on HW cache-coherent protocol instead of SW interrupts



Fast Resource Scaling Mechanism

2. Shallow sleep with instant transition --- **How core enters/exits sleep?**

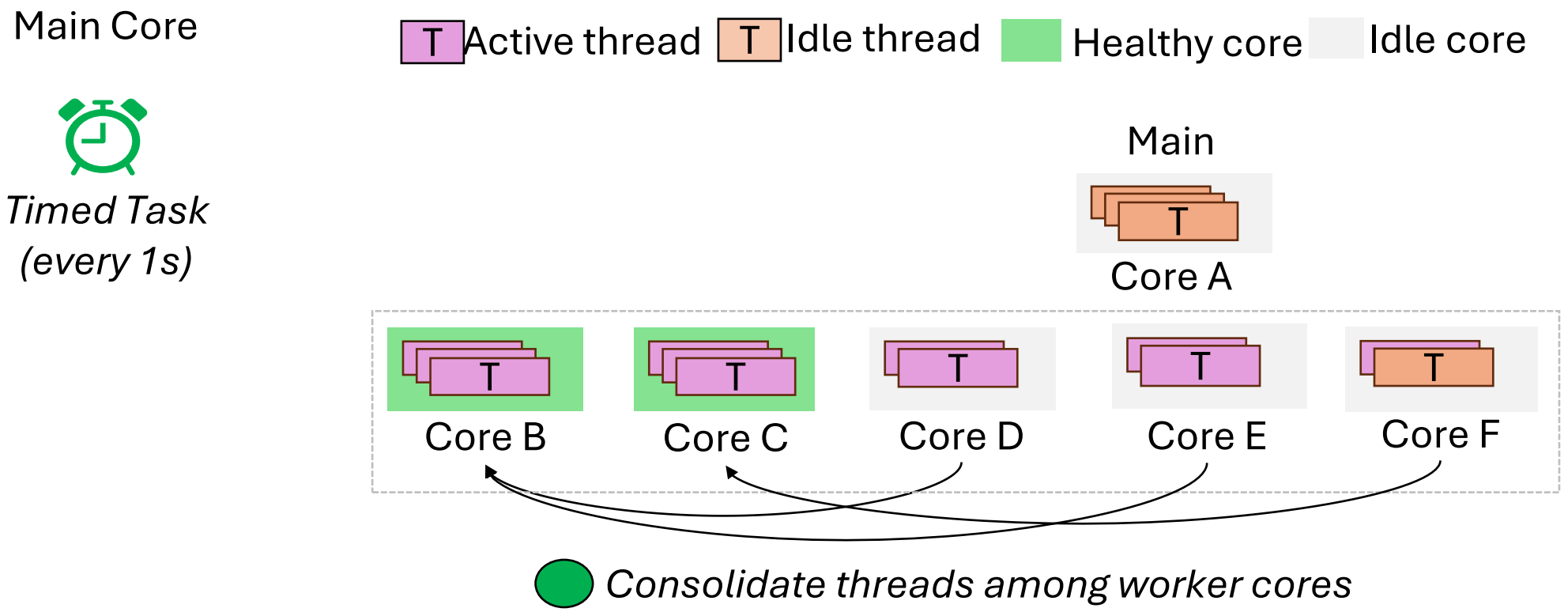
- **Goal**: wake up fast without overhead
- Wake up core based on HW cache-coherent protocol instead of SW interrupts



Scheduling Flow Overview

Two granularities of scheduling --- **When to put cores to sleep and wake up?**

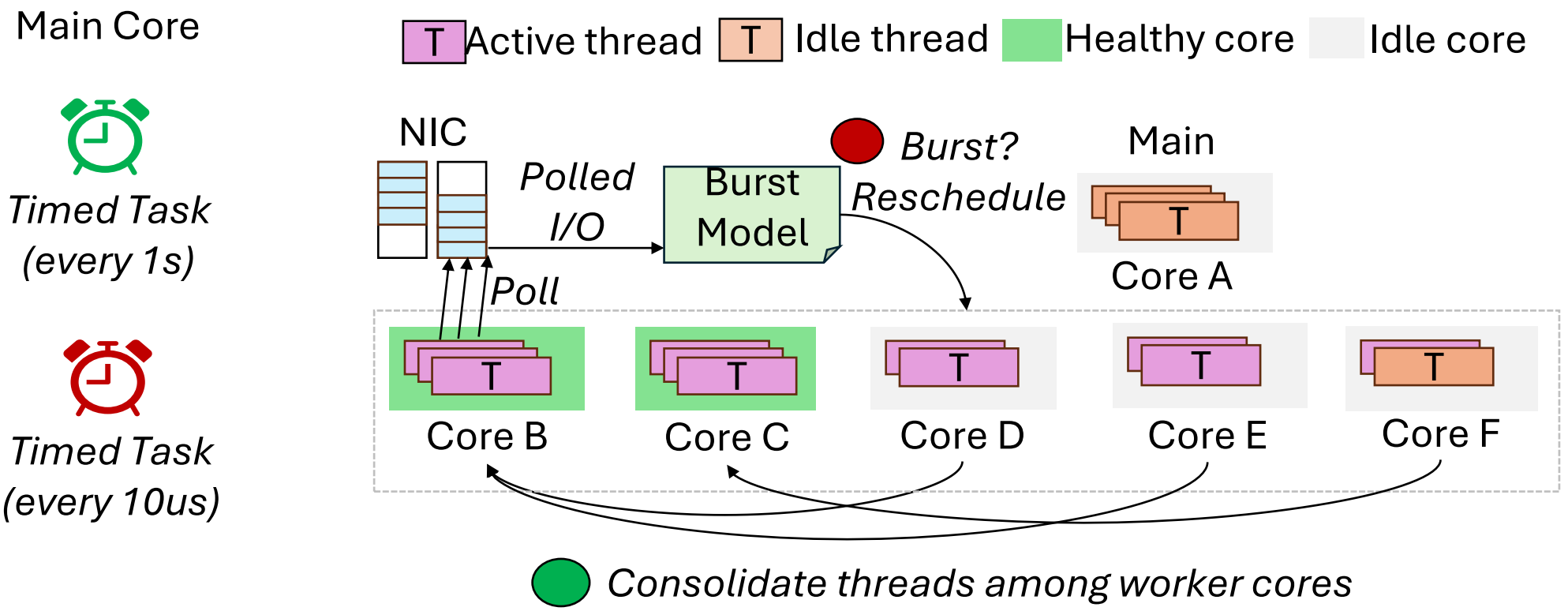
- Coarse-grained (1s): check wasted CPU cycles to decide whether to **reduce # of cores**



Scheduling Flow Overview

Two granularities of scheduling --- **When to put cores to sleep and wake up?**

- Coarse-grained (1s): check wasted CPU cycles to decide whether to **reduce # of cores**
- Fined-grained (10us): detect bursts from NIC queues to decide to **allocate more cores**



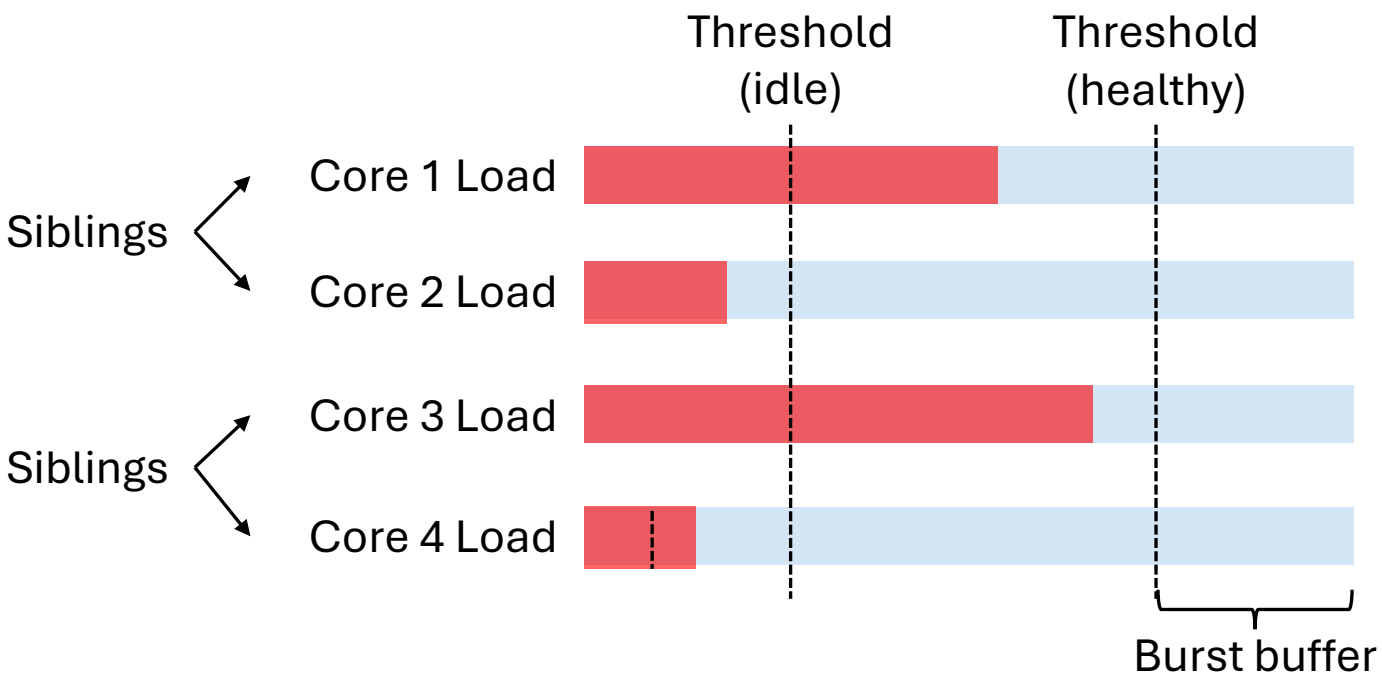
Monitoring Resource Wastage

Which cores to put sleep?

Monitoring Resource Wastage

Which cores to put sleep?

- **Consolidate threads of idle cores into healthy cores**

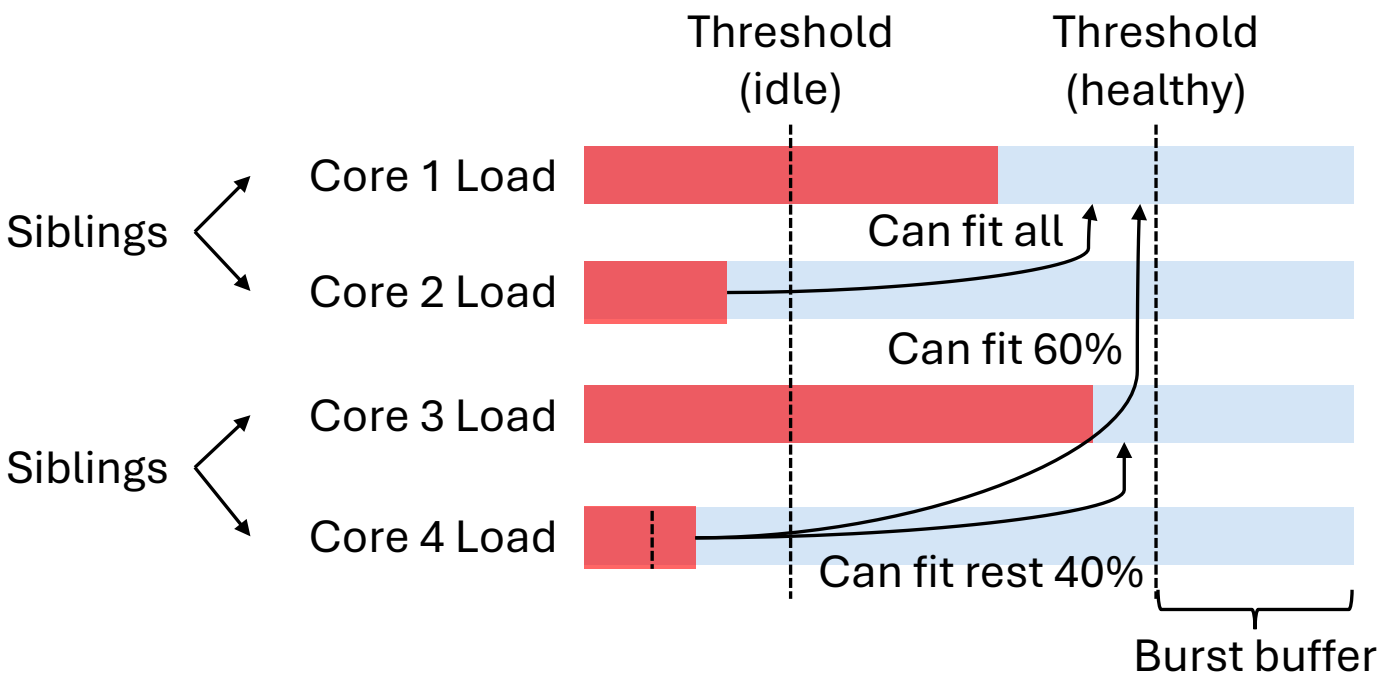


Reserve burst buffer (CPU cycles) shared by all threads on the same core for handling bursts

Monitoring Resource Wastage

Which cores to put sleep?

- **Consolidate threads of idle cores into healthy cores**



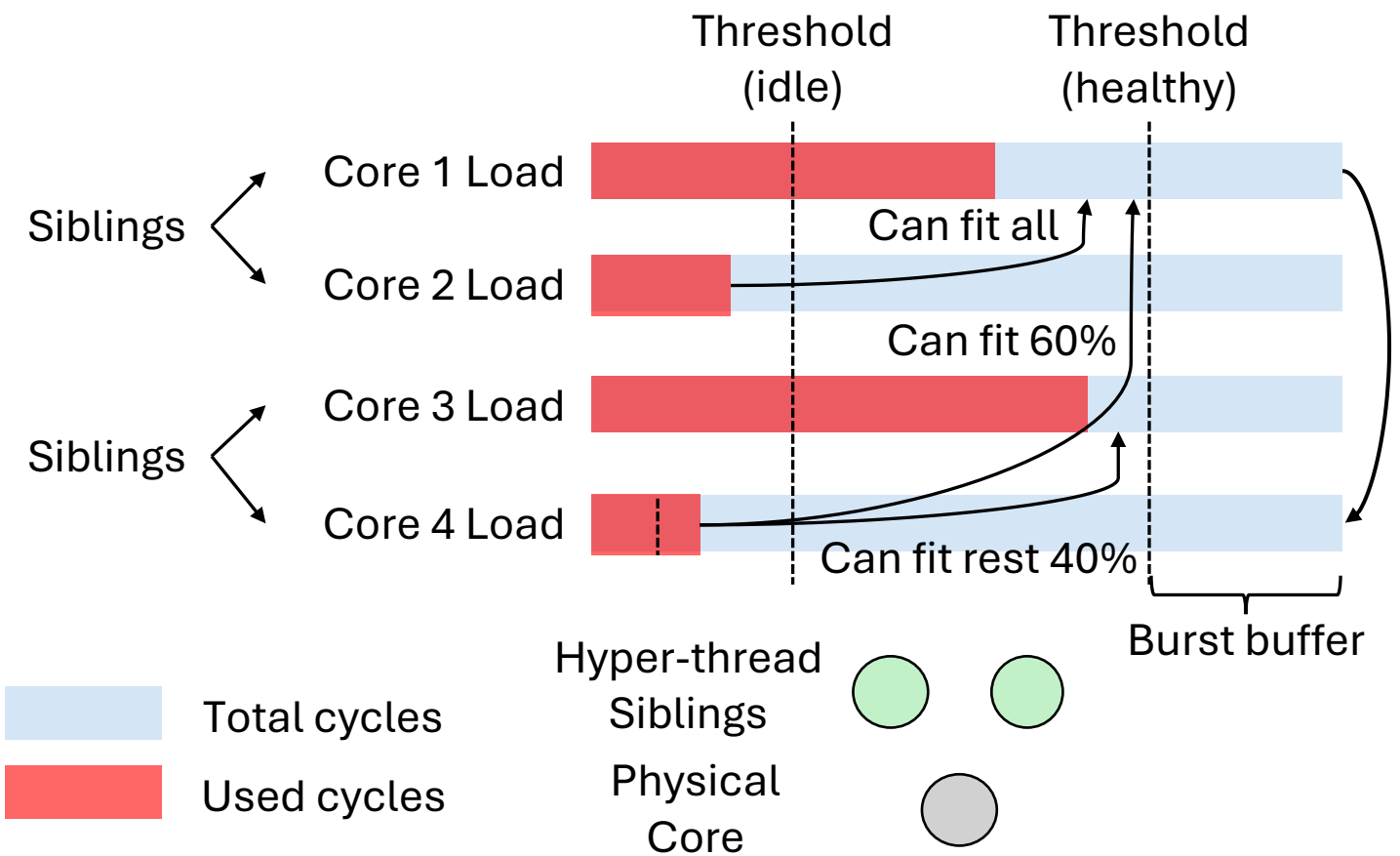
Reserve burst buffer (CPU cycles) shared by all threads on the same core for handling bursts

Check whether core can fit thread based on thread load and core load

Monitoring Resource Wastage

Which cores to put sleep?

- Consolidate threads of idle cores into healthy cores
- **Schedule sibling cores to sleep together**



Reserve burst buffer (CPU cycles) shared by all threads on the same core for handling bursts

Check whether core can fit thread based on thread and core load

Move threads of an active core to another free core if its sibling is idle and ready to sleep

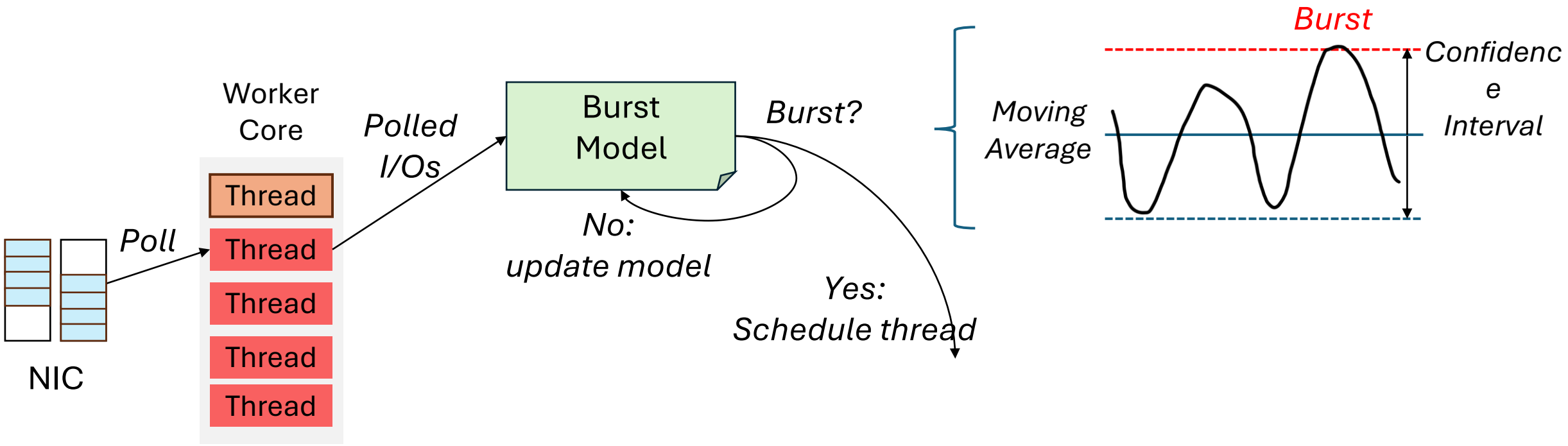
Detecting I/O Bursts

Which cores to wake up?

Detecting I/O Bursts

Which cores to wake up?

- **Spread out threads that require more compute resource**

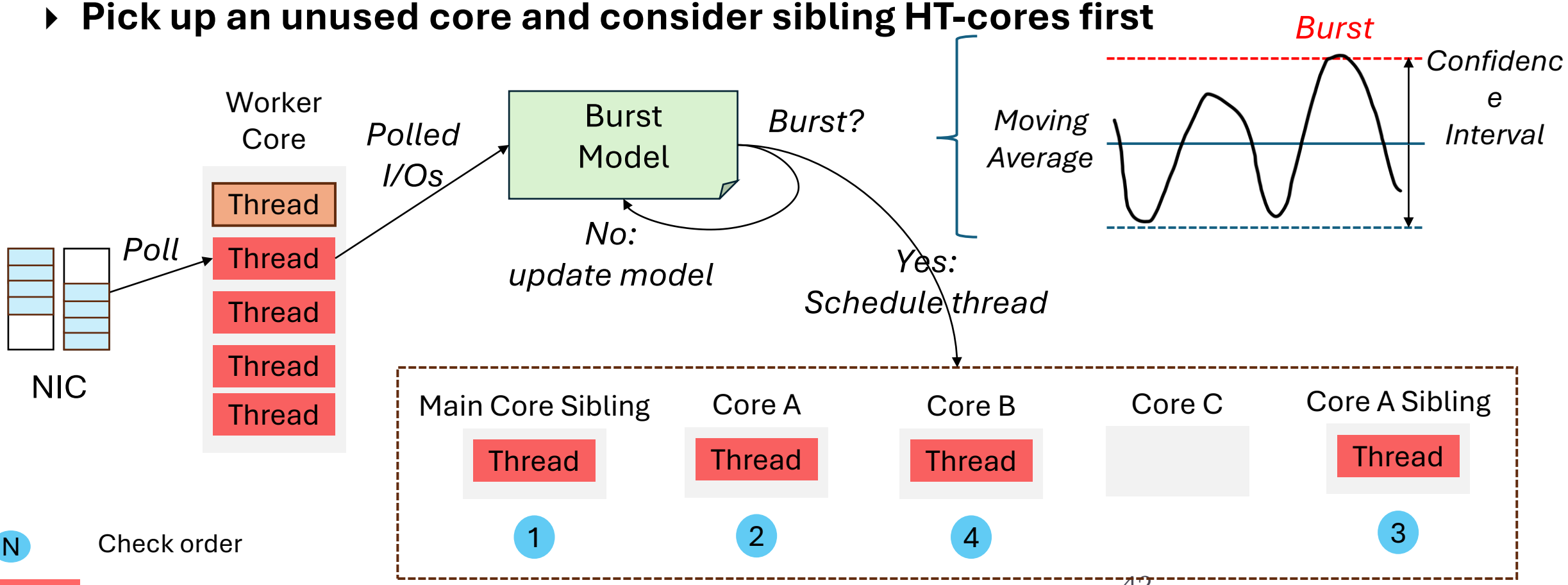


Thread Thread to be scheduled

Detecting I/O Bursts

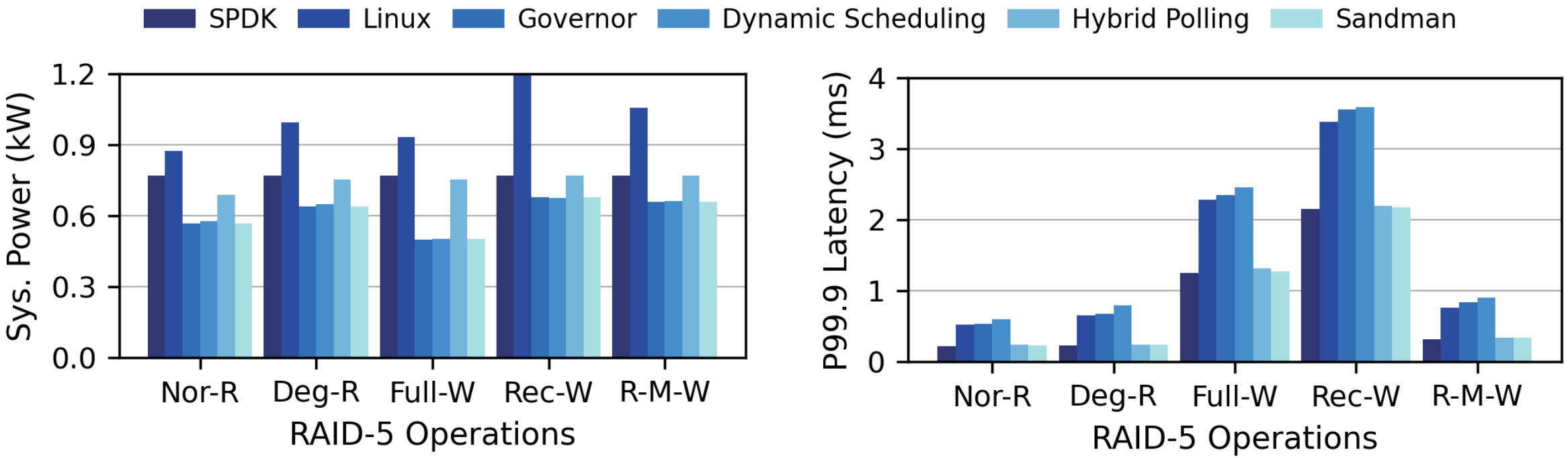
Which cores to wake up?

- Spread out threads that require more compute resource
- **Pick up an unused core and consider sibling HT-cores first**



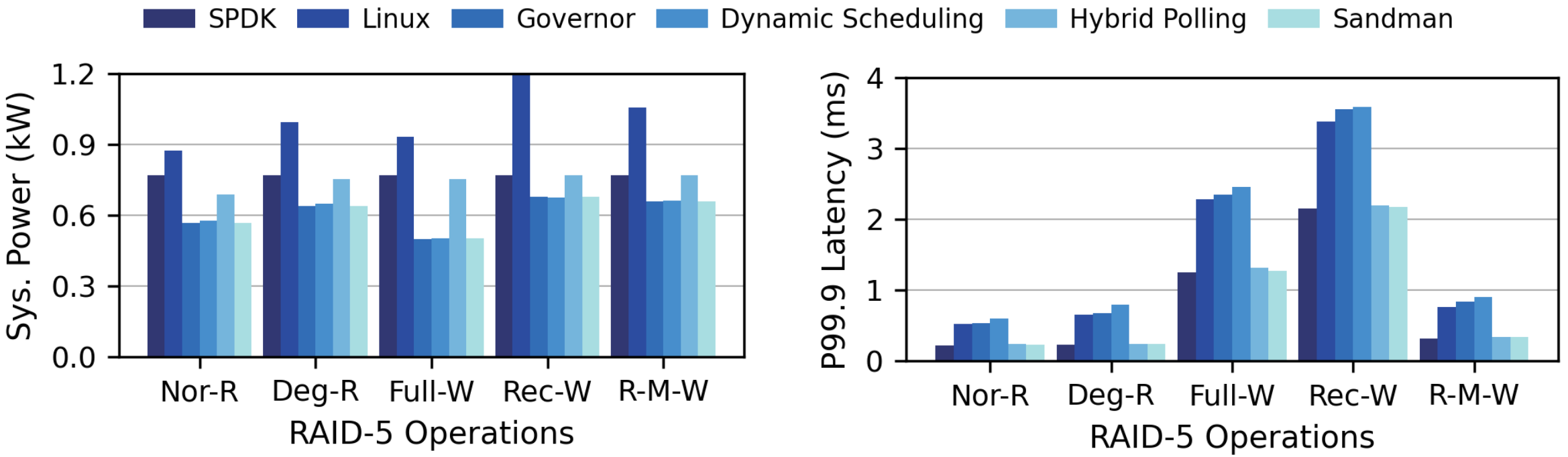
Application Benefits

Flash array configured with RAID-5



Application Benefits

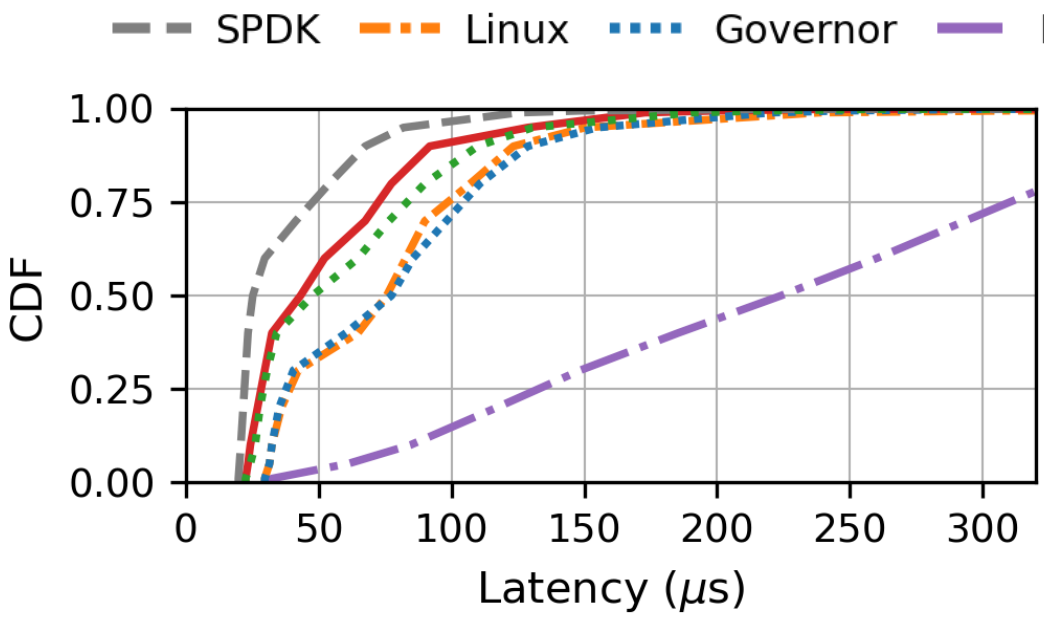
Flash array configured with RAID-5



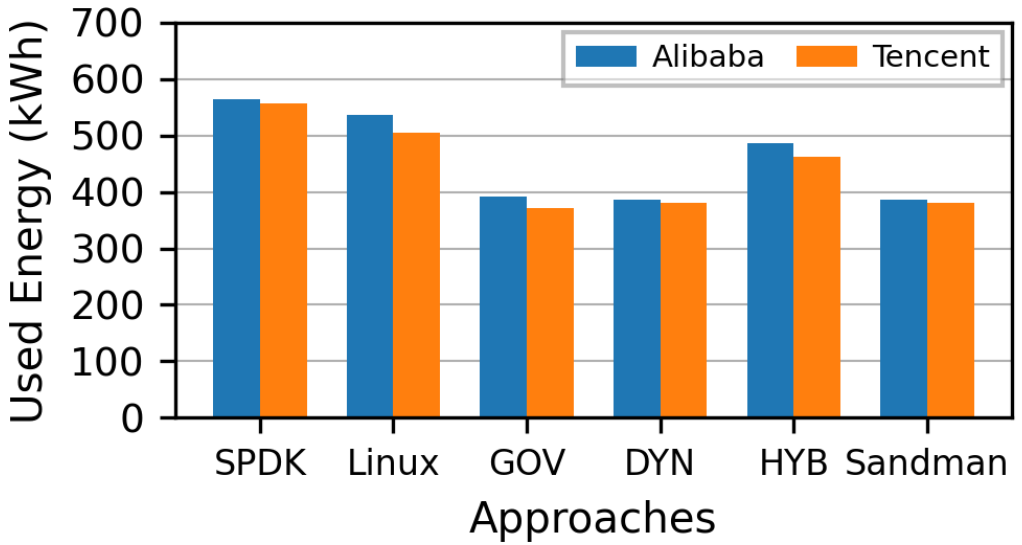
Sandman achieves comparable latency to SPDK but reduces power consumption **up to 39.38%** under flash array with RAID-5

Benefits under Cloud Workloads

Replaying block-level traces from public cloud EBS products



Latency CDF under Alibaba's Trace



Monthly Energy Consumption

Sandman achieves similar latency as SPDK while reducing energy consumption ***33.36% vs. SPDK*** under cloud workloads

Takeaways

- Storage trends to consume more energy with evolving towards high performance and high density
- Current storage stacks either consume more power or sacrifice performance
- Storage stacks can be both high performance and energy efficiency with a carefully designed scheduler with hardware assistance.

Thank You!

Q & A

More details are in the paper